

Platforms like Amazon Web Services, Google Cloud Platform, and Microsoft Azure introduced fully managed infrastructure services.

Managed databases, object storage, auto-scaling, and global failover -- all these abstracted away. Dev teams stopped thinking about servers (infrastructure management) and started thinking about features and application logic. For many companies, this was genuinely transformative.

And now? Self-hosting is making a comeback: Honestly, it makes sense. Cloud costs have increased a lot at scale. Data regulatory and data sovereignty laws have gotten stricter. And open source tooling has matured to the point where a small team can run and build sophisticated infrastructure without relying on cloud providers.

Let's understand self-hosting infrastructure

Self-hosting means your organisation controls the infrastructure. That could be:

Bare metal: Physical servers, usually for high-performance workloads like ML or large databases.

VPS: Virtual machines from providers like Hetzner or DigitalOcean (hardware's theirs, OS is yours).

On-premises: Your own data centre, common in banking, healthcare, and government.

Self-managed Kubernetes: Using tools like K3s or Rancher to run containerised workloads with cloud-like orchestration, but on your own infrastructure.

Here are a few reasons why teams prefer self-hosting.

Full control over the stack: You can tune your PostgreSQL config, install extensions like PostGIS or TimescaleDB, and configure custom networking. Cloud providers don't let you do that. This matters a lot for teams with specific requirements.

Data privacy and compliance: If you're dealing with GDPR, HIPAA, or

any regulation that cares about where data lives, self-hosting simplifies things significantly. Your data stays in environments you control. No external service provider in the chain.

Predictable costs: Cloud hosting services usually operate on a usage-based billing model. It sounds flexible until you get an unpredictable invoice. Self-hosting is mostly capex. You buy hardware or sign infrastructure contracts upfront. This often works out for stable, high-traffic workloads and is considerably cheaper over time.

Open source alignment: There's a solid ecosystem here. Proxmox for virtualization, Ceph for distributed storage, Prometheus and Grafana for monitoring, Traefik for ingress, and Ansible for config management. You can build a serious stack without relying on a single proprietary service.

However, self-hosting still has many drawbacks.

Maintenance is a real challenge: Security patches, OS upgrades, hardware failures, performance monitoring — someone must do all of this. If there is no strong automation, it can consume most of the engineering time.

Scaling isn't automatic: Cloud environments scale on demand. In self-hosted setups, you're provisioning servers or virtual machines manually when the load increases. This requires planning and lead time.

Security is entirely on you: Your team needs to manage firewalls, encryption, access controls, vulnerability, etc. Security tools like Fail2Ban, Trivy, and Falco help, but they need proper configuration and someone who knows them well.

You need DevOps expertise: This is where small teams often get stuck. Hiring DevOps engineers who understand distributed systems, Kubernetes, and networking is expensive. Small teams can't afford the high cost of hiring these experts, so they sometimes avoid using them.

Managed cloud: The other side

Managed cloud flips the entire work model on which self-hosting works. Here, you interact with higher-level services such as managed databases, object storage services, serverless computing platforms, and managed Kubernetes clusters, etc, and the provider handles everything underneath.

Examples of managed cloud services include Amazon RDS, Google Kubernetes Engine, AWS Lambda, and S3. You don't touch OS updates, hardware replacement, or network configs. Instead, the provider does.

The advantages of managed cloud are:

Deployment speed: New services go live fast, without any procurement or hardware setup.

Auto-scaling: If traffic spikes, the platform handles it. It is critical for consumer apps with unpredictable loads.

Reduced ops burden: Small teams can run complex infrastructure without dedicated DevOps headcount.

Built-in redundancy: Most providers run across multiple availability zones with 99.9%+ uptime SLAs.

Ecosystem integrations: Cloud providers have pre-built versions of almost everything — ML tools, analytics platforms, messaging systems.

There are a few disadvantages too.

- Vendor lock-in is a major drawback. If your app is deeply tied to provider-specific APIs, migrating is painful and expensive.
- Pricing is hard to predict. Costs may depend on compute usage, storage consumption, network bandwidth, and data transfer. Data egress charges can increase operational costs.
- Customisation has limits because managed cloud services are specially designed for common use cases. Kernel-level tuning, custom networking setups, and specialised hardware access are often not possible in managed environments. Open source technologies play a vital role in closing the gap between